Counterfactual Payment Risk Engine

Step-by-step build plan, module by module

PayPal AI Hackathon - system-level AI/ML project

What this document is	A practical, ordered procedure for building the whole system: what each module does, how to build it, how to check it works, and what usually goes wrong.
The idea in one line	A real-time decision layer that does not just say approve or decline: it finds the smallest change that makes a risky transaction acceptable, and learns the cheapest friction to apply.
Assumptions	48-hour hackathon, Python on Ubuntu, a public dataset (IEEE-CIS or PaySim), 1 to 4 people. A 24-hour cut list is included.

Contents

Contents	2
1. Project at a Glance	3
2. Environment and Repository Setup	4
3. Module M1 - Data and Features	5
4. Module M2 - Detector and Calibration	7
5. Module M3 - Graph and Relationship Features	8
6. Module M4 - Counterfactual Engine	9
7. Module M5 - Friction Policy (Contextual Bandit)	11
8. Module M6 - Adversary Loop	13
9. Module M7 - Serving API	14
10. Module M8 - Dashboard	15
11. Module M9 - Evaluation and Results	16
12. Build Schedule	17
13. Risks and Mitigations	18
14. Pitch and Demo Plan	19
Appendix A - Definition of Done Checklist	20

1. Project at a Glance

Most risk systems output one score and a binary decision. This project adds three things on top of a strong detector: a **counterfactual module** that answers "what would have to change for this to be safe?", a **learned friction policy** (contextual bandit) that picks the cheapest effective intervention, and an **adversarial loop** where an attacker agent keeps probing the detector so it gets harder to evade.

1.1 Architecture

```
raw transaction
       |
       v
[M1 Data + Features] --> [M2 Detector + Calibration] --> risk r
       |                           ^                        |
       v                           | retrain                |
[M3 Graph features]       [M6 Adversary loop]              v
                                                  [M4 Counterfactual search]
                                                           |
[M5 Friction policy (bandit)] <-- context + risk ----------+
       |
       v
action --> [M7 API: analyst / user view] --> [M8 Dashboard] --> [M9] --> [M10]
```

1.2 Module map

Module	Purpose	Est. hours	Priority
M1 Data and features	Load data, time-based split, causal features	4	Must
M2 Detector	LightGBM + probability calibration + metrics	4	Must
M3 Graph features	Shared-entity signals (device, email, address)	3 (+6 for GNN)	Should
M4 Counterfactual engine	Minimal actionable change that flips the decision	6	Must
M5 Friction policy	Simulator + contextual bandit vs baselines	6	Must
M6 Adversary loop	Attacker agent + adversarial retraining	5	Should
M7 Serving API	FastAPI with analyst and user views	3	Must
M8 Dashboard	Streamlit demo of the full flow	5	Must
M9 Evaluation	Experiments, ablations, results table	4	Must
M10 Pitch	Slides, demo script, Q and A prep	3	Must

Total is about 43 person-hours. With 3 to 4 people this fits comfortably in 48 hours; solo, drop M6 and the GNN part of M3 first.

1.3 Honest framing (decide this on day one)

- Public datasets contain **no record of interventions** (step-up, hold) or their outcomes. The friction policy therefore runs in a **simulated environment**. Say so clearly in the pitch; it is a strength when stated, a weakness when discovered.
- Counterfactuals describe the **model's** decision boundary, not causal truth about fraud. Phrase them as "the model would accept this if...".
- Never let an LLM make the risk decision. If you add one, use it only to turn evidence into readable explanations.

2. Environment and Repository Setup

2.1 Machine setup (Ubuntu)

```
sudo apt update && sudo apt install -y python3-venv git build-essential
mkdir payrisk && cd payrisk && git init
python3 -m venv .venv && source .venv/bin/activate
pip install --upgrade pip
pip install pandas numpy scikit-learn lightgbm shap fastapi uvicorn \
    streamlit matplotlib plotly pyarrow networkx pydantic kaggle
mkdir -p data/raw data/processed src/{data,models,counterfactual,policy,adversary,api} \
    dashboard notebooks reports tests
pip freeze > requirements.txt
```

2.2 Get the data

- **IEEE-CIS Fraud Detection** (recommended): real e-commerce transactions, about 590k rows, about 3.5% fraud. Needs a Kaggle account, accepting the competition rules, and an API token in `~/.kaggle/kaggle.json`.
- **PaySim** (fallback): synthetic mobile-money data with readable column names, but it is very easy to separate and less realistic.
- Command: `kaggle competitions download -c ieee-fraud-detection -p data/raw`, then unzip.

2.3 Repository layout and conventions

```
payrisk/
  data/raw  data/processed
  src/data/            load.py  features.py
  src/models/          detector.py  metrics.py
  src/counterfactual/  search.py
  src/policy/          simulator.py  bandit.py  baselines.py
  src/adversary/       attacker.py  loop.py
  src/api/             main.py
  dashboard/           app.py
  reports/             figures + results.json
  config.yaml          thresholds, costs, seeds
```

- One **config.yaml** holds thresholds, simulator costs and random seeds so every experiment is reproducible.
- One **make_features()** function used by training, serving, counterfactuals and the attacker (train-serve parity).
- Commit after each module. Tag a working checkpoint at every milestone in section 12.

3. Module M1 - Data and Features

Goal: a clean, time-ordered dataset with causal features (each row sees only the past). **Output:** four parquet files (train, val, calib, test) and a reusable `make_features()`.

Steps

- Merge `train_transaction.csv` with `train_identity.csv` on `TransactionID` (left join).
- Sort by `TransactionDT` and split **by time**: 60% train, 15% val (early stopping), 10% calib (calibration), 15% test. Never shuffle.
- Add history features once on the sorted frame: per-card prior count, prior mean amount, time since last transaction.
- Add per-row features: hour, day of week, log amount, amount relative to the card's prior mean.
- Cast categoricals using categories learned on train only; store those category lists for serving.

```
# src/data/load.py
import pandas as pd

def load_ieee(path="data/raw"):
    tx = pd.read_csv(f"{path}/train_transaction.csv")
    idn = pd.read_csv(f"{path}/train_identity.csv")
    df = tx.merge(idn, on="TransactionID", how="left")
    return df.sort_values("TransactionDT").reset_index(drop=True)

def time_split(df, fracs=(0.60, 0.15, 0.10, 0.15)):
    out, start, n = [], 0, len(df)
    for f in fracs:
        end = start + int(n * f)
        out.append(df.iloc[start:end].copy())
        start = end
    return out          # train, val, calib, test
```

```
# src/data/features.py
import numpy as np

def add_history(df):                        # run ONCE on the time-sorted frame
    df["uid"] = df["card1"].astype(str) + "_" + df["addr1"].astype(str)
    g = df.groupby("uid")
    df["prior_cnt"] = g.cumcount()
    prior_sum = g["TransactionAmt"].cumsum() - df["TransactionAmt"]
    mean = prior_sum / df["prior_cnt"].replace(0, np.nan)
    df["prior_mean_amt"] = mean.fillna(df["TransactionAmt"].median())
    df["since_last"] = g["TransactionDT"].diff().fillna(-1)
    return df

def make_features(df):                      # used everywhere, row-wise safe
    df = df.copy()
    df["hour"] = (df["TransactionDT"] // 3600) % 24
    df["dow"] = (df["TransactionDT"] // 86400) % 7
    df["log_amt"] = np.log1p(df["TransactionAmt"])
    df["amt_ratio"] = df["TransactionAmt"] / (df["prior_mean_amt"] + 1e-6)
    return df
```

Done when

- Every val/calib/test row has `TransactionDT` later than every train row (assert this in a test).
- Fraud rate per split is printed and is similar (about 3 to 4%).
- `make_features(single_row_df)` equals the same row from `make_features(full_df)` (parity test).

Pitfalls

- `TransactionDT` is a time offset in seconds from an unknown reference, so "hour of day" is approximate. It is a common community convention; mention it as an assumption.

- Most IEEE-CIS columns are masked (V1 to V339, C, D, M). Only a few are interpretable, which directly shapes M4 (see its actionable-feature design).
- Never compute history features separately per split; compute once on the full sorted frame so test rows see train history, as in real life.

4. Module M2 - Detector and Calibration

Goal: a LightGBM classifier whose output is a trustworthy probability. Calibration matters because M4 and M5 treat the score as a real risk number. **Output:** `risk_fn(X) -> probabilities`, metrics report, calibration plot.

```
# src/models/detector.py
import lightgbm as lgb
from sklearn.isotonic import IsotonicRegression

def fit_detector(Xtr, ytr, Xva, yva, Xcal, ycal, seed=42):
    model = lgb.LGBMClassifier(
        objective="binary", learning_rate=0.05, num_leaves=64, n_estimators=3000,
        subsample=0.8, subsample_freq=1, colsample_bytree=0.7,
        min_child_samples=50, random_state=seed)
    model.fit(Xtr, ytr, eval_set=[(Xva, yva)], eval_metric="auc",
              callbacks=[lgb.early_stopping(100), lgb.log_evaluation(200)])
    iso = IsotonicRegression(out_of_bounds="clip")
    iso.fit(model.predict_proba(Xcal)[:, 1], ycal)       # separate calib slice

    def risk_fn(X):
        return iso.predict(model.predict_proba(X)[:, 1])
    return model, risk_fn
```

```
# src/models/metrics.py
import numpy as np
from sklearn.metrics import average_precision_score, roc_auc_score, roc_curve

def recall_at_fpr(y, p, target=0.01):
    fpr, tpr, _ = roc_curve(y, p)
    return float(np.interp(target, fpr, tpr))

def ece(y, p, bins=15):
    y, p = np.asarray(y), np.asarray(p)
    idx = np.minimum((p * bins).astype(int), bins - 1)
    total = 0.0
    for b in range(bins):
        m = idx == b
        if m.any():
            total += abs(y[m].mean() - p[m].mean()) * m.mean()
    return total

def report(y, p):
    return dict(pr_auc=average_precision_score(y, p), roc_auc=roc_auc_score(y, p),
                recall_at_1pct_fpr=recall_at_fpr(y, p), ece=ece(y, p))
```

Steps

- Train the baseline with all numeric columns plus categoricals as `category` dtype.
- Evaluate on test: PR-AUC, ROC-AUC, recall at 1% FPR, ECE. Plot a reliability diagram (before and after isotonic).
- Pick thresholds `TAU_ALLOW` (below this, allow) and `TAU_BLOCK` by minimizing expected cost on the validation set; store them in config.yaml.
- Save feature importance and SHAP values for the analyst view in M7 (`model.booster_.predict(X, pred_contrib=True)`).

Done when

- Test metrics are saved to `reports/results.json`. Expect roughly 0.88 to 0.93 ROC-AUC with a clean time-based split; do not chase leaderboard numbers.
- The reliability diagram sits close to the diagonal after calibration.

> **Note.** Early stopping uses the val slice and calibration uses a different slice. Using the same slice for both leaks information and makes the calibration look better than it is.

5. Module M3 - Graph and Relationship Features

Goal: catch rings and shared infrastructure that a single-row model misses. Start with cheap, causal graph-style features; treat a real GNN as a stretch goal. **Output:** extra columns appended in `add_history()` and an ablation showing their value.

Level 1: shared-entity features (3 hours)

Treat device, email domain and billing address as entity nodes connected to card identities. For each row compute how many events and how many **distinct cards** have touched that entity **so far**.

```
ENTITY_COLS = ["DeviceInfo", "P_emaildomain", "R_emaildomain", "addr1"]

def add_graph_feats(df):                    # df sorted by time, has df["uid"]
    for col in ENTITY_COLS:
        df[f"{col}_prior_cnt"] = df.groupby(col).cumcount().astype(float)
        # running count of distinct uids seen on this entity (causal, O(n))
        pair_new = (~df.duplicated([col, "uid"])).astype(int)
        distinct = pair_new.groupby(df[col]).cumsum()
        df[f"{col}_distinct_uids"] = distinct
        df[f"{col}_is_new"] = (df[f"{col}_prior_cnt"] == 0).astype(int)
    graph_cols = [c for c in df.columns if c.endswith(("_prior_cnt", "_distinct_uids"))]
    df[graph_cols] = df[graph_cols].fillna(0)   # missing entity -> no graph signal
    return df
```

Level 2 (stretch): GraphSAGE embeddings (+6 hours)

- Build a bipartite graph: card-identity nodes and entity nodes (device, email, address); edges are transactions.
- Use PyTorch Geometric `SAGEConv`, two layers, 64-dim output, unsupervised or supervised on train-period edges only.
- Export one embedding per card identity as extra features for LightGBM. Build the graph snapshot from the train period only, otherwise future edges leak.
- For the demo, draw the suspicious cluster with networkx or plotly: it makes a strong visual.

Done when

- Ablation table: detector with and without graph features (PR-AUC and recall at 1% FPR).
- At least one example flagged mainly because of a shared device or email cluster, ready for the demo.

6. Module M4 - Counterfactual Engine

Goal: for a transaction the detector flags, return the smallest, most plausible changes that would bring the risk below `TAU_ALLOW`. This is the core differentiator. **Output:** `counterfactuals(x, risk_fn, tau) -> ranked list of (cost, new_risk, changed_transaction)`.

6.1 Design: actionable versus immutable

A counterfactual is only useful if the change is something a user or system can actually do. Because most IEEE-CIS columns are masked, define a small, interpretable, controllable set:

Feature changed	Real-world meaning	Allowed range
`TransactionAmt`	Pay part now, the rest after verification	10% to 100% of original
`TransactionDT` (hour)	Schedule the payment for later	0 to 24 hours delay
`card6`	Pay with a different funding source	debit or credit
Everything else	Immutable (device, account age, history, graph signals)	Never changed

6.2 Search algorithm

- Enumerate a grid of candidates over the actionable features (6 amount levels x 7 delays x 2 methods = 83 candidates).
- Recompute every derived feature for each candidate with the shared `make_features()`.
- Score all candidates in one batch call to the detector (fast; one predict call).
- Keep candidates with risk below `TAU_ALLOW`, rank by a change cost, and return the top k, preferring diverse changes (one amount-based, one delay-based, one method-based).

```
# src/counterfactual/search.py
import itertools
import numpy as np
import pandas as pd

AMT_MULTS = [1.0, 0.9, 0.75, 0.5, 0.25, 0.1]
DELAYS_H = [0, 1, 2, 4, 8, 12, 24]
CARD6_OPTS = ["debit", "credit"]
W = dict(amt=1.0, delay=0.5, method=0.3)       # tune: how "expensive" each change is

def change_cost(x, y):
    amt = abs(np.log(y["TransactionAmt"] / x["TransactionAmt"]))
    delay_days = (y["TransactionDT"] - x["TransactionDT"]) / 86400
    method = float(y["card6"] != x["card6"])
    return W["amt"] * amt + W["delay"] * delay_days + W["method"] * method

def candidates(x):
    for m, d, c in itertools.product(AMT_MULTS, DELAYS_H, CARD6_OPTS):
        if m == 1.0 and d == 0 and c == x["card6"]:
            continue                           # this is the original transaction
        y = dict(x)
        y["TransactionAmt"] = x["TransactionAmt"] * m
        y["TransactionDT"] = x["TransactionDT"] + d * 3600
        y["card6"] = c
        yield y

def counterfactuals(x, risk_fn, tau, k=3):
    cands = list(candidates(x))
    feats = make_features(pd.DataFrame(cands))  # recompute derived features
    risks = risk_fn(feats)
    valid = [(change_cost(x, y), float(r), y)
             for y, r in zip(cands, risks) if r < tau]
    return sorted(valid, key=lambda t: t[0])[:k]
```

6.3 Upgrades if time allows

- Use `dice-ml` (`pip install dice-ml`) with `features_to_vary` and `permitted_range` for a library-backed version, and compare it with your grid on validity and proximity.
- Replace the grid with a genetic or CMA-ES search over continuous amount and delay values.
- Add a plausibility filter: reject candidates whose amount ratio or hour is far outside the card's historical range.

6.4 Metrics to report

Metric	Definition
Validity	Share of flagged transactions for which at least one counterfactual crosses below the threshold
Proximity	Mean `change_cost` of the best counterfactual (lower is better)
Sparsity	Mean number of features changed (1 or 2 is ideal)
Plausibility	Share of counterfactuals inside the card's historical amount and time range

Pitfalls

- **Derived-feature drift:** if you change the amount but forget to recompute `log_amt` and `amt_ratio`, the counterfactual is meaningless. Always go through `make_features()`.
- **Category mismatch:** after building the candidate DataFrame, cast `card6` back to the training categories or LightGBM will raise or mis-code values.
- **Boundary leakage:** showing exact counterfactuals to end users lets attackers map your decision boundary. Analysts see detail; users see coarse guidance (see M7).

7. Module M5 - Friction Policy (Contextual Bandit)

Goal: choose among ALLOW, STEP_UP, HOLD, BLOCK to minimize total cost = fraud loss + friction + lost legitimate revenue. **Output:** a simulator, a LinUCB agent, and a cost comparison against baselines.

7.1 The simulator (state its assumptions openly)

```
# src/policy/simulator.py
ACTIONS = ["ALLOW", "STEP_UP", "HOLD", "BLOCK"]
SIM = {                                  # all values are configurable assumptions
    "ALLOW":   dict(fric=0.0, catch=0.00, abandon=0.00),
    "STEP_UP": dict(fric=1.0, catch=0.85, abandon=0.03),
    "HOLD":    dict(fric=3.0, catch=0.95, abandon=0.10),
    "BLOCK":   dict(fric=5.0, catch=1.00, abandon=1.00),
}
MARGIN = 0.03          # revenue kept on a legit transaction, as a share of amount

def realized_cost(action, is_fraud, amount, rng, sim=SIM):
    s = sim[action]
    cost = s["fric"]
    if is_fraud:
        if rng.random() > s["catch"]:
            cost += amount                # fraud slips through: full loss
    elif rng.random() < s["abandon"]:
        cost += MARGIN * amount           # legit user lost
    return cost

def expected_cost(action, r, amount, sim=SIM):
    s = sim[action]
    return (s["fric"] + r * (1 - s["catch"]) * amount
            + (1 - r) * s["abandon"] * MARGIN * amount)
```

7.2 Baselines (the comparison is your headline result)

Policy	Rule
B1 Single threshold	Block if risk above `TAU_BLOCK`, else allow
B2 Static ladder	Fixed risk bands: allow, step-up, hold, block
B3 Analytic policy	Pick the action minimizing `expected_cost` using the simulator values (upper bound when the world is stationary)
Ours: LinUCB bandit	Learns from observed costs only; adapts when the world changes

Key experimental design. If the simulator is known and fixed, the analytic policy B3 is already optimal, so a bandit cannot beat it. The bandit earns its place when the world changes. Make the simulator **non-stationary**: halfway through the stream, drop the STEP_UP catch rate from 0.85 to 0.40 (attackers learned to pass OTP). B1, B2 and a stale B3 keep choosing STEP_UP; the bandit shifts to HOLD. That divergence is your plot.

7.3 LinUCB agent

```
# src/policy/bandit.py
import numpy as np

class LinUCB:
    def __init__(self, n_actions, d, alpha=1.0):
        self.A = [np.eye(d) for _ in range(n_actions)]
        self.b = [np.zeros(d) for _ in range(n_actions)]
        self.alpha = alpha

    def choose(self, x):
        scores = []
        for A, b in zip(self.A, self.b):
            theta = np.linalg.solve(A, b)
            width = np.sqrt(x @ np.linalg.solve(A, x))
            scores.append(theta @ x + self.alpha * width)
        return int(np.argmax(scores))

    def update(self, a, x, reward):
        self.A[a] += np.outer(x, x)
        self.b[a] += reward * x
```

```
# context: keep it small and scaled
def context(row, r):
    la = np.log1p(row.TransactionAmt)
    return np.array([1.0, r, la / 10, r * la / 10,
                     np.sin(2*np.pi*row.hour/24), np.cos(2*np.pi*row.hour/24),
                     min(row.addr1_distinct_uids, 20) / 20])

def run_policy(df, risk, choose, update, rng, sim_at):
    total, curve = 0.0, []
    for i, row in enumerate(df.itertuples()):       # replay test split as a stream
        x = context(row, risk[i])
        a = choose(x)
        c = realized_cost(ACTIONS[a], row.isFraud, row.TransactionAmt, rng, sim_at(i))
        update(a, x, -min(c, 500.0) / 100.0)        # clip and scale the reward
        total += c
        curve.append(total)
    return np.array(curve)
```

Steps

- Implement the simulator and baselines first and plot their cost curves on a stationary stream.
- Add LinUCB and tune `alpha` on the validation stream (try 0.3, 1.0, 2.0).
- Switch on non-stationarity and produce the headline plot: cumulative cost over time for all policies, with a vertical line where the world changes.
- Report total cost, fraud loss, friction cost, and legit-user loss separately in a stacked bar.

Pitfalls

- Bandits see only the cost of the action they took. That partial feedback is the point, but make sure the code never leaks the counterfactual costs of other actions into the update.
- Fraud losses are heavy tailed. Clip or winsorize rewards and scale them, or one huge transaction will dominate learning.
- If LinUCB underperforms, add interaction features (risk x log amount, already included) before changing the algorithm.

8. Module M6 - Adversary Loop

Goal: show that the detector becomes harder to evade after training against an attacker. **Output:** an attacker agent and a plot of evasion rate across retraining rounds.

8.1 Threat model (write this down first)

- Black-box attacker: can query the score with a budget of Q queries per attempt.
- Controls only the same actionable fields as M4 (amount, timing, funding method), and must keep at least half the original amount so the fraud stays profitable.
- Success (evasion) means the final risk is below `TAU_ALLOW`.

```
# src/adversary/attacker.py
import numpy as np
import pandas as pd

def mutate(x, rng, min_amt_frac=0.5):
    y = dict(x)
    y["TransactionAmt"] = x["TransactionAmt"] * rng.uniform(min_amt_frac, 1.0)
    y["TransactionDT"] = x["TransactionDT"] + int(rng.integers(0, 24)) * 3600
    y["card6"] = rng.choice(CARD6_OPTS)
    return y

def attack(x, risk_fn, rng, budget=50, min_amt_frac=0.5):
    cands = [mutate(x, rng, min_amt_frac) for _ in range(budget)]
    risks = risk_fn(make_features(pd.DataFrame(cands)))     # one batch of queries
    i = int(np.argmin(risks))
    return cands[i], float(risks[i])
```

8.2 Adversarial retraining loop

```
# src/adversary/loop.py  (pseudocode close to final code)
for rnd in range(ROUNDS):                       # e.g. 5 rounds
    evaded = []
    for x in fraud_rows(val):                   # generate attacks on VALIDATION fraud
        y, r = attack(x, risk_fn, rng, budget=Q)
        if r < TAU_ALLOW:
            evaded.append(y)
    evasion_test = measure_evasion(fraud_rows(test), risk_fn, Q)   # report on TEST fraud
    log(rnd, evasion_test, clean_metrics=report(y_test, risk_fn(X_test)))
    train_df = pd.concat([train_df, to_frame(evaded, label=1)])
    model, risk_fn = fit_detector(*prepare(train_df))              # reuse M2 code
```

8.3 Plots to produce

- Evasion rate on test fraud versus retraining round (should fall).
- Clean PR-AUC versus round (should not collapse; if it does, down-weight adversarial rows).
- Evasion rate versus query budget Q, with and without a defense (rate limiting or coarse score output). This ties directly into the boundary-leakage point in M4 and M7.

> **Note.** Perturbations are applied in feature space, which is a simplification: real attackers face stricter constraints. State this as a limitation and a next step.

9. Module M7 - Serving API

Goal: one HTTP endpoint that returns a decision, with different detail for analysts and end users. **Output:** a FastAPI app you can call from the dashboard and from curl.

```
# src/api/main.py
from fastapi import FastAPI
from pydantic import BaseModel
import pandas as pd

app = FastAPI()

class Tx(BaseModel):
    TransactionAmt: float
    TransactionDT: int
    card6: str
    prior_mean_amt: float
    # ... plus the other model input fields, with defaults for optional ones

@app.get("/health")
def health():
    return {"ok": True}

@app.post("/decide")
def decide(tx: Tx, view: str = "user"):
    x = tx.dict()
    X = make_features(pd.DataFrame([x]))
    r = float(risk_fn(X)[0])
    a = policy.choose(context_from(x, r))
    resp = {"action": ACTIONS[a]}
    if view == "analyst":
        resp["risk"] = r
        resp["top_reasons"] = top_shap(X, k=5)
        resp["counterfactuals"] = counterfactuals(x, risk_fn, TAU_ALLOW)
    else:
        resp["guidance"] = coarse_guidance(ACTIONS[a])   # no thresholds, no numbers
    return resp
```

Steps

- Load the model, calibrator, config and the saved bandit once at startup, not per request.
- Return exact scores and counterfactuals only to the analyst view. The user view returns an action and a coarse message such as "additional verification needed".
- Add simple per-client rate limiting to support the defense experiment in M6.
- Run with `uvicorn src.api.main:app --reload` and test with `curl -X POST localhost:8000/decide?view=analyst -H 'Content-Type: application/json' -d @sample.json`.

Done when

- p95 latency of the decision path (without the counterfactual search) is comfortably under 100 ms on your laptop; the counterfactual search is analyst-only.
- Five saved sample transactions (clean, suspicious, ring-based, large, borderline) return sensible, repeatable results.

LLM explanation layer (optional)

If you add an LLM, pass it only the structured evidence (top SHAP reasons, counterfactuals, chosen action) and ask it to write a short analyst summary. The decision is already made by M2 to M5; the LLM never changes it.

10. Module M8 - Dashboard

Goal: a fast, reliable demo UI. Streamlit is the quickest route on Ubuntu; use React only if you already have a front-end person. **Output:** `dashboard/app.py`.

Page	Shows	Source
1 Live decision	Pick a sample transaction; risk, action, top reasons	POST /decide (analyst)
2 Counterfactual explorer	Sliders for amount and delay; risk updates live; the ranked "yes, if..." options	risk_fn directly or API
3 Policy battle	Cumulative cost of all policies, vertical line at the drift point	Precomputed results
4 Adversary arena	Evasion rate versus round; clean PR-AUC versus round	Precomputed results
5 Network view (if M3 done)	Suspicious cluster graph for a flagged card	networkx or plotly

Steps

- Precompute pages 3 and 4 as JSON/parquet in `reports/` so the demo never waits for training.
- Keep pages 1 and 2 live; they are the interactive moment judges remember.
- Run with `streamlit run dashboard/app.py`. Use `st.cache_resource` for the model and `st.cache_data` for result files.
- Record a 90-second screen capture of the full flow as a fallback in case Wi-Fi or hardware fails on demo day.

11. Module M9 - Evaluation and Results

Goal: a single results table you can paste into the README and slides, backed by reproducible scripts.

ID	Experiment	Compare	Headline output
E1	Detector quality	LightGBM vs logistic regression	PR-AUC, recall at 1% FPR
E2	Calibration	Raw vs isotonic	ECE and reliability plot
E3	Graph value	With vs without M3 features	PR-AUC delta
E4	Counterfactual quality	Grid vs DiCE (if built)	Validity, proximity, sparsity
E5	Policy under drift	B1, B2, B3, bandit	Cumulative cost curve and cost breakdown
E6	Adversarial robustness	Before vs after rounds	Evasion rate and clean PR-AUC
E7	Defense	Open vs rate-limited or coarse output	Evasion versus query budget

Fairness and safety check (brief but include it)

- Do not use protected attributes as features. Compare false-positive rates across a few available segments (for example product type or region proxy) and report any large gaps.
- State that real deployment would start in shadow mode, then an A/B test with real friction outcomes, and would use off-policy evaluation (inverse propensity or doubly robust) to evaluate policies on logged data.

Reproducibility

- Fixed seeds in `config.yaml`. One script per experiment (`python -m src.experiments.e5_policy`) writing to `reports/results.json`.
- A `README.md` with setup, data download, and the exact commands to regenerate every figure.

12. Build Schedule

12.1 The 48-hour plan with checkpoints

Hours	Work	Checkpoint (must be demo-able)
0-2	Setup, repo, data download, config	Data loads in a notebook
2-8	M1 features and splits, M2 detector and calibration	**A (H8):** baseline metrics saved
8-12	M3 level-1 graph features, ablation	Graph delta measured
12-20	M4 counterfactual engine and metrics	**B (H20):** CLI shows flagged tx and options
20-30	M5 simulator, baselines, LinUCB, drift experiment	**C (H30):** policy cost plot exists
30-36	M6 attacker and retraining loop	Evasion-versus-round plot
36-40	M7 API with analyst and user views	**D (H40):** API returns full decision
40-44	M8 dashboard, fallback video	Full flow runs end to end
44-46	M9 final numbers, README, cleanup	Results table frozen
46-48	M10 slides, rehearsal x2	**E:** pitch rehearsed with timer

Note. Reserve the last four hours for no new features: only bug fixes, rehearsal and the fallback video.

12.2 Splitting work across a team of 3

Person	Owns	Hands off to others
A	M1, M2, M3, M7 (data, detector, API)	`risk_fn` and `make_features()` on day one
B	M4, M6 (counterfactual, adversary)	Counterfactual results and plots
C	M5, M8 (policy, dashboard)	Needs `risk_fn` early; use a stub until A delivers
All	M9 and M10	Each owns the experiments for their modules

12.3 The 24-hour cut list

- Drop in this order: GNN, M6 adversary loop, DiCE comparison, network view, LLM explanations.
- Keep: M1, M2, M4 grid search, M5 with the drift experiment, M7 minimal API, a 3-page dashboard, and one results table.
- That core alone is a complete, defensible submission.

13. Risks and Mitigations

Risk	Impact	Mitigation
Time leakage in features or splits	Inflated metrics, credibility loss	Time-based splits, causal-feature tests, assert statements
Masked dataset columns	Hard to explain counterfactuals	Restrict actionable set to amount, timing, method; say so
Simulated friction outcomes	Judges question realism	Disclose, make parameters configurable, show drift robustness, propose shadow-mode validation
Bandit does not beat analytic policy	Weak headline	Design the drift experiment; report honestly where each wins
Counterfactuals leak decision boundary	Attackers exploit them	Analyst-only detail, coarse user guidance, rate limits, E7 experiment
Scope creep (GNN, LLM, React)	Nothing finishes	Follow the cut list and checkpoints strictly
Demo failure on stage	Lost impact	Precomputed pages, fallback video, local-only dependencies
Training time on 590k rows	Slow iteration	Cache parquet, subsample to 100k while developing, full run overnight

14. Pitch and Demo Plan

14.1 Five-minute structure

Time	Beat	Content
0:00	Problem	Risk systems say no with no recourse; legit users are blocked and attackers probe the black box
0:30	Insight	Decisions should be "yes, if..." with the cheapest friction, learned over time
1:00	Live demo	Flag a transaction, show the analyst view, counterfactual options, chosen action, and the user's coarse view
2:30	Result 1	Policy under drift: bandit versus static rules
3:30	Result 2	Adversary loop: evasion falls across rounds, with the defense curve
4:15	Honesty	Simulated friction, feature-space attacker, shadow-mode rollout plan
4:45	Close	"Today's risk systems say no. Ours says yes, if... and learns the cheapest way to get there."

14.2 Likely judge questions

Question	Short answer
Why not just tune a threshold?	A threshold cannot adapt when attackers change tactics; the E5 drift plot shows static rules degrade while the bandit recovers.
Is the simulator realistic?	It is an explicit assumption with configurable parameters. Production would log real interventions and use off-policy evaluation before any rollout.
Do counterfactuals help attackers?	Detail is analyst-only; users get coarse guidance, and E7 measures evasion versus query budget with and without defenses.
Is this just fraud detection?	No. Detection is one module; the contribution is the decision layer (counterfactuals, learned friction, adversarial hardening), which applies to any high-stakes transaction.
What about fairness?	No protected attributes; segment-level false-positive checks; human review path for borderline cases.

14.3 Roadmap slide

- Shadow-mode deployment, then A/B testing with real step-up and hold outcomes.
- Graph neural network embeddings and streaming feature store (Kafka and Redis) for production scale.
- Causal methods and off-policy evaluation to replace the simulator.

Appendix A - Definition of Done Checklist

Module	Done when
M1	Time-ordered splits asserted; parity test passes; features cached to parquet
M2	Test metrics saved; reliability diagram near diagonal; thresholds in config
M3	Ablation table produced; one ring example ready for demo
M4	Validity, proximity, sparsity reported; CLI prints options for any flagged row
M5	Drift experiment plot with 4 policies; cost breakdown bar chart
M6	Evasion-versus-round plot; defense curve; clean PR-AUC tracked
M7	Analyst and user views behave differently; five sample requests saved
M8	Full flow runs from a clean start in under a minute; fallback video recorded
M9	One results table; one command per experiment; README complete
M10	Slides finished; two timed rehearsals; judge Q and A rehearsed

Appendix B - config.yaml starter

```
seed: 42
split: {train: 0.60, val: 0.15, calib: 0.10, test: 0.15}
thresholds: {tau_allow: 0.10, tau_block: 0.85}      # set from validation cost
counterfactual:
  amt_mults: [1.0, 0.9, 0.75, 0.5, 0.25, 0.1]
  delays_h: [0, 1, 2, 4, 8, 12, 24]
  weights: {amt: 1.0, delay: 0.5, method: 0.3}
policy:
  margin: 0.03
  linucb_alpha: 1.0
  drift_at: 0.5                    # fraction of stream where STEP_UP catch drops
  step_up_catch_after: 0.40
adversary:
  rounds: 5
  query_budget: 50
  min_amt_frac: 0.5
```